

5. ТЕСТИРОВАНИЕ И РАЗВЕРТЫВАНИЕ СИСТЕМЫ

5.1 Модульное и интеграционное тестирование компонентов

Тестирование является неотъемлемой частью разработки любой программной системы, особенно микросервисной, где ошибка в одном компоненте может затронуть работу всей платформы. В проекте реализовано несколько уровней тестирования, охватывающих отдельные классы, группы компонентов и сквозные пользовательские сценарии [3].

Для модульного тестирования используется связка JUnit 5 и Mockito. Модульные тесты проверяют корректность работы отдельных классов в изоляции от внешних зависимостей. Например, в сервисе поездок тестируется метод `createRide` сервисного слоя: репозитории и Feign-клиенты подменяются mock-объектами, после вызова метода проверяется, что на репозитории был вызван метод `save` с корректными параметрами, а в Feign-клиент ушли правильные запросы. Mockito позволяет задать ожидаемое поведение mock-объектов и проверить факт их вызова, при этом реальные HTTP-запросы и операции с БД не выполняются. Модульные тесты выполняются быстро и не требуют запущенной инфраструктуры, поэтому их удобно прогонять при каждой сборке проекта [3].

Интеграционные тесты проверяют взаимодействие компонентов с реальной инфраструктурой. Для их выполнения используется библиотека TestContainers, которая позволяет программно запускать Docker-контейнеры с PostgreSQL и Kafka непосредственно из тестового кода. Тест, помеченный аннотацией `@Testcontainers`, при запуске создаёт временный контейнер PostgreSQL с чистой БД, накатывает на неё Liquibase-миграции, и далее тест работает с БД так же, как реальное приложение. После завершения теста контейнер уничтожается вместе с данными. Аналогично для Kafka поднимается временный контейнер, в котором создаются топики с тестовыми именами, и тесты проверяют корректность отправки и получения сообщений [3,8].

В сервисе поездок интеграционные тесты проверяют полный цикл работы с заказом. Сначала тест создаёт поездку через контроллер: передаётся запрос с адресами и идентификатором пассажира. Контроллер обращается к сервисному слою, тот обращается к Marbox-клиенту (который для тестов также подменяется mock-объектом, возвращающим заранее заданные координаты и дистанцию), после чего создаётся запись `Ride` в БД. Тест проверяет, что поездка сохранена с правильными полями, статус равен `WAITING_FOR_DRIVER`, а в таблице `kafka_messages` появилась запись с топиком `passenger-busy-topic`. Затем через контроллер выполняется назначение водителя, и тест проверяет смену статуса и появление записи для `driver-busy-topic`. После этого статус меняется несколько раз, и тест на каждом шаге проверяет корректность перехода с помощью методов конечного автомата.

Для тестирования пользовательских сценариев в проекте используется подход BDD (Behavior-Driven Development) с фреймворком Cucumber. Сценарии описываются на естественном языке в feature-файлах с помощью ключевых слов `Given`, `When`, `Then`. Например, сценарий "Создание поездки пассажиром" выглядит так: `Given` существует активный пассажир; `When` пассажир создаёт поездку с адресами "ул. Московская, 267" и "ул. Советская, 50"; `Then` поездка создана, статус равен `WAITING_FOR_DRIVER`, и пассажир поме-

					ДП.AC64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		5

чен как занятый. За каждым шагом закреплён Java-метод, реализующий соответствующее действие или проверку. Cucumber-тесты запускаются вместе с интеграционными и проверяют те же пользовательские сценарии, которые описаны в функциональных требованиях [3].

Для тестирования API-эндпоинтов применяется библиотека REST Assured. Она предоставляет удобный DSL для отправки HTTP-запросов и проверки ответов. В тесте формируется запрос с телом в формате JSON, отправляется на заданный URL, затем проверяется код ответа (например, 201 при создании или 200 при успешном запросе), наличие определённых полей в ответе и их значения. REST Assured позволяет тестировать эндпоинты на уровне HTTP, не поднимая весь HTTP-сервер, что ускоряет выполнение тестов. В связке с TestContainers это даёт возможность протестировать полный путь от HTTP-запроса до записи в БД.

Для контроля качества кода в проекте настроен Checkstyle. Конфигурация задана в файле checkstyle.xml в корне проекта и основана на Google Java Style с адаптацией под принятые в проекте соглашения. Checkstyle проверяет форматирование кода, наличие неиспользуемых импортов, длину строк и методов, соответствие Javadoc-комментариев. Проверка запускается при сборке Maven и не позволяет закоммитить код, не соответствующий стандартам. Это поддерживает единый стиль кода во всех сервисах, даже если над ними работает несколько человек [4].

Все тесты запускаются командой `mvn verify` в корневом `pom.xml` проекта. Maven последовательно проходит по всем модулям и в каждом выполняет компиляцию, модульные тесты, интеграционные тесты и проверку Checkstyle. Если любой тест не проходит или Checkstyle находит нарушения, сборка считается неудачной. Такой подход гарантирует, что любое изменение кода проходит полную проверку перед тем, как попасть в основную ветку проекта.

5.2 Контейнеризация приложения с использованием Docker

Развёртывание микросервисной системы, состоящей из восьми сервисов, пяти БД и нескольких вспомогательных служб, вручную было бы трудоёмким и чреватым ошибками. Для автоматизации этого процесса все компоненты системы упакованы в Docker-контейнеры. Docker позволяет запустить весь проект одной командой, гарантируя, что все сервисы работают в одинаковом окружении независимо от операционной системы хоста [1].

Сборка Docker-образа для каждого сервиса описана в `Dockerfile`, расположенном в директории `docker`. Файл использует многоступенчатую сборку. На первом этапе (`build stage`) используется образ Maven с JDK 17. Исходный код сервиса копируется в контейнер, и выполняется команда `mvn package -DskipTests`, собирающая исполняемый JAR-файл. На втором этапе создаётся минимальный образ на базе JRE. В него копируется только собранный JAR-файл, после чего указывается команда запуска `java -jar`. Благодаря разделению на этапы итоговый образ содержит только среду выполнения и скомпилированный код, без Maven, исходников и промежуточных артефактов. Это уменьшает размер образа и ускоряет его загрузку [1].

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		6

Для сборки конкретного сервиса используется плагин `spring-boot-maven-plugin`, который формирует JAR-файл со встроенным сервером. В результате каждый сервис представляет собой самодостаточный исполняемый модуль, не требующий установки внешнего сервера приложений. Все зависимости, включая Tomcat, уже включены в JAR.

Оркестрация контейнеров описана в файле `docker-compose.yml`. Этот файл задаёт состав системы, связи между компонентами и их конфигурацию. В него включены все восемь сервисов системы: `config-server`, `eureka-server` и `api-gateway` как инфраструктурная основа, а также `auth-service`, `driver-service`, `passenger-service`, `ride-service` и `rating-service` как бизнес-логика. Для каждого сервиса указан свой Dockerfile, порт, переменные окружения и политика перезапуска.

Помимо сервисов, `docker-compose.yml` описывает вспомогательные контейнеры. Для каждой бизнес-БД поднимается отдельный экземпляр PostgreSQL. Всего их четыре: `driver_db`, `passenger_db`, `ride_db` и `rating_db`. Для каждого контейнера заданы имя БД, пользователь и пароль через переменные окружения `POSTGRES_DB`, `POSTGRES_USER` и `POSTGRES_PASSWORD`. Бизнес-сервисы подключаются к своей БД, используя эти учётные данные, переданные в конфигурации сервиса.

Контейнер Redis используется для кэширования геоданных в сервисе поездок. Контейнер Kafka работает в режиме KRaft, не требующем отдельного ZooKeeper. Kafka UI предоставляет веб-интерфейс для просмотра топиков, сообщений и потребителей. Контейнер Keycloak обеспечивает аутентификацию и авторизацию, его БД также развёрнута в отдельном контейнере PostgreSQL. PGAdmin предоставляет графический интерфейс для управления всеми БД.

Для локальной разработки предусмотрен упрощённый файл `docker-compose-local.yml`. Он поднимает только инфраструктурные компоненты - Redis, Kafka, Keycloak и БД, - но не запускает сами бизнес-сервисы. Разработчик запускает сервисы локально в IDE, а они подключаются к инфраструктуре из контейнеров. Это ускоряет цикл разработки, так как не требуется пересобирать Docker-образы после каждого изменения кода.

Конфигурационные настройки для всех окружений вынесены в файл `.env` в директории `docker`. В нём определены переменные: пароли для БД, учётные данные администратора Keycloak, API-ключ Marbox, адреса Kafka-брокеров и флаги включения мониторинга. Файл `.env` подключается в `docker-compose.yml`, и его значения подставляются в переменные окружения контейнеров. При развёртывании на новом хосте достаточно изменить значения в `.env` без правки `compose`-файла.

Для удобства управления в корне проекта создан Makefile. Команда `make up` запускает полное окружение описанное в `docker-compose.yml`. Команда `make down` останавливает и удаляет все контейнеры. Команда `make up-local` запускает облегчённое окружение из `docker-compose-local.yml`. Такой подход избавляет от необходимости запоминать длинные `docker compose` команды с путями к файлам.

Благодаря Eureka, сервисы автоматически находят друг друга при запуске. После того как `eureka-server` поднялся и принял регистрации от остальных сервисов, `api-gateway` начинает маршрутизировать запросы, а Feign-клиенты находят целевые сервисы по именам. Никакой ручной настройки адресов не требуется - вся маршрутизация строится динамически на основе данных из реестра сервисов [1].

5.3 Настройка CI/CD процессов и мониторинг серверной части

После развёртывания системы необходимо обеспечить наблюдение за её состоянием. В распределённой микросервисной архитектуре, где запрос пользователя может пройти через несколько сервисов, традиционные подходы к мониторингу недостаточны - требуется видеть не только факт ошибки, но и её причину в цепочке вызовов. Для решения этих задач в проекте используется стек технологий наблюдаемости, включающий сбор метрик, трассировку запросов и агрегацию логов [2].

Сбор метрик реализован с помощью Prometheus. Каждый микросервис предоставляет HTTP-эндпоинт `/actuator/prometheus`, отдающий метрики в формате, понятном Prometheus. Для включения этого эндпоинта в каждом сервисе добавлены зависимости `spring-boot-actuator` и `micrometer-registry-prometheus`. Prometheus периодически опрашивает все сервисы и сохраняет полученные метрики в своей БД временных рядов. Среди собираемых метрик: количество HTTP-запросов (с разбивкой по методам и кодам ответа), время обработки запросов в различных перцентилях, использование памяти JVM, загрузка процессора, количество активных соединений с БД, статистика по Kafka-сообщениям [2].

Визуализация метрик выполняется в Grafana. Для проекта подготовлены дашборды, отображающие ключевые показатели каждого сервиса: количество запросов в секунду, долю ошибочных ответов, среднее и максимальное время ответа. Отдельный дашборд показывает состояние JVM: использование heap-памяти, активность сборщика мусора, количество потоков. Дашборд для Kafka отображает количество сообщений в топиках, задержку доставки и статистику потребителей. Все дашборды обновляются в реальном времени, что позволяет оперативно обнаруживать аномалии в работе системы [2].

Для распределённой трассировки запросов используется Tempo. Когда пользовательский запрос приходит на API Gateway, ему присваивается уникальный `trace_id`. Этот идентификатор передаётся всем нижестоящим сервисам через HTTP-заголовки, а при отправке сообщений в Kafka сохраняется в полях `trace_id` и `span_id` таблицы `kafka_messages` и восстанавливается на стороне потребителя. Благодаря этому в Tempo можно проследить полный путь запроса: какие сервисы были задействованы, сколько времени заняла обработка на каждом из них, на каком именно шаге произошла ошибка. Трассировка особенно полезна при отладке проблем с производительностью, когда нужно найти узкое место в длинной цепочке вызовов [2,5].

Экспорт трассировок в Tempo осуществляется через протокол OTLP (OpenTelemetry Protocol). В каждом сервисе добавлены зависимости для OpenTelemetry, а в конфигурации указан адрес Tempo-коллектора. Spring Boot автоматически инструментирует входящие и исходящие HTTP-запросы, добавляя к ним `trace`-заголовки. Для Kafka-сообщений инструментирование реализовано вручную в коде отправителя и получателя, что позволяет сохранять сквозную трассировку даже при асинхронном взаимодействии.

Агрегация логов выполняется с помощью связки Loki и Promtail. Promtail запускается как `sidecar`-контейнер и отслеживает лог-файлы всех сервисов. Он собирает записи, добавляет к ним метки с именем сервиса, и отправляет в Loki. Loki хранит логи и предоставляет API для их поиска. В Grafana логи отображаются рядом с метриками и трассировками, что даёт полную картину работы системы в одном интерфейсе. Например, при обнаружении всплеска ошибок через метрики Prometheus можно сразу перейти к логам со-

ответствующего сервиса и найти точное исключение, а затем по `trace_id` проследить весь путь запроса в Tempo [2].

Для управления версиями БД в процессе разработки и развёртывания используется Liquibase. Миграции хранятся в XML-файлах и описывают изменения схемы для каждой версии продукта. При старте сервиса Liquibase автоматически проверяет, какие миграции уже применены, и выполняет только новые. Это гарантирует, что схема БД всегда соответствует версии кода, как на локальной машине разработчика, так и в продуктивном окружении [6].

Процесс непрерывной интеграции строится вокруг Maven. При каждом изменении кода выполняется команда `mvn clean verify`, которая последовательно запускает компиляцию всех модулей, модульные тесты, интеграционные тесты и проверку Checkstyle. Если любой из этапов завершается неудачно, сборка считается проваленной, и разработчик получает уведомление. Такой подход гарантирует, что в основную ветку проекта попадает только код, прошедший полную автоматическую проверку [3].

Отдельного упоминания заслуживают механизмы отказоустойчивости, также являющиеся частью эксплуатации системы. Это ShedLock для предотвращения дублирования Kafka-сообщений при запуске нескольких экземпляров `ride-service`, Resilience4J для защиты от каскадных отказов при межсервисных вызовах и Outbox-паттерн для гарантированной доставки событий. Все эти механизмы были подробно рассмотрены в главе 4 при описании реализации соответствующих модулей [1,8].

					ДП.АС64.220047-05 81 00	Лист
						9
Изм.	Лист	№ докум.	Подп.	Дата		